

Universidad del Valle de Guatemala
Facultad de Ingeniería



Laboratorio 8

Integrantes

Ronald Manuel Catún Chacón - 19789

PROGRAMACIÓN DE MICROPROCESADORES
- SECCIÓN 10 -

Ciudad de Guatemala, Guatemala
2026

Ejercicio I

Matriz rectangular de 400x600 elementos aleatorios. Cada hilo ordena sus filas en orden descendente y calcula/imprime el valor máximo de cada fila. Se utilizan 4 hilos; cada uno recibe un struct con puntero a la matriz, filas totales, columnas totales e índices de inicio y fin.

Preguntas del Ejercicio I

1. ¿Qué ventajas tiene dividir la matriz por filas frente a dividirla por bloques de filas y columnas?

Dividir por filas completas es más simple y eficiente. Cada hilo trabaja sobre filas enteras contiguas en memoria, aprovechando la localidad de caché (los arrays de C++ almacenan filas de forma contigua). Cada hilo es completamente autónomo sobre sus filas y no necesita sincronización con otros. Si en cambio se divide por bloques (rectángulos de filas y columnas), un hilo debe saltar por posiciones de memoria no contiguas, incrementando los cache misses y la complejidad de los índices. Además, para ordenar una fila completa se necesitaría coordinar entre los hilos que la comparten, introduciendo la necesidad de mutexes o barreras.

2. ¿Qué cambios habría que hacer si cada hilo tuviera que ordenar columnas en lugar de filas?

El cambio principal está en cómo se accede a los datos. En la implementación actual, ordenar una fila es trivial porque los elementos son contiguos en memoria. Para ordenar columnas habría que:

- Extraer la columna j : iterar sobre todas las filas $\text{matriz}[0][j]$, $\text{matriz}[1][j]$, ..., $\text{matriz}[N-1][j]$ en un vector temporal.
- Ordenar ese vector temporal y escribir de vuelta los valores a la columna original.
- El struct ThreadData cambiaría: en lugar de inicio/fin de filas, tendría inicio/fin de columnas.
- El rendimiento se degradaría notablemente porque el acceso por columnas no es contiguo en memoria (stride access), generando muchos cache misses.

Ejercicio II

El usuario tiene que ingresar el tamaño del arreglo y la cantidad de hilos. El programa valida que $n \geq \text{num_hilos}$; si no, muestra error y repite la solicitud. Cada hilo calcula la suma parcial y la cantidad de números pares de su segmento. El hilo principal reúne los resultados e imprime la suma total y el total de pares.

Preguntas del Ejercicio II

1. ¿Qué parte del programa habría que modificar para que cada hilo busque el valor máximo de su segmento en vez de contar números pares?

Únicamente se modifican dos partes del código:

- El struct: cambiar el campo `int cantPares` por `int maximo` (inicializado en `INT_MIN` o en el primer elemento del segmento).
- La función del hilo: dentro del bucle, en lugar del contador de pares, se hace: `if (val > data->maximo) data->maximo = val;`
- El main: en la reunión de resultados, en lugar de sumar `cantPares`, se comparan los máximos parciales para obtener el máximo global.

La lógica de distribución de índices, creación/join de hilos y el struct no necesitan cambios estructurales — solo se sustituye el cómputo interno.

2. ¿Por qué usar demasiados hilos no siempre mejora el rendimiento del programa?

Hay varios factores que limitan la ganancia de añadir más hilos:

- Overhead de creación y sincronización: cada `pthread_create` y `pthread_join` tiene un costo fijo. Si el trabajo por hilo es pequeño, este overhead supera la ganancia.
- Límite de núcleos físicos: en una CPU de 4 núcleos, 4 hilos corren en verdadero paralelo; los adicionales compiten por los mismos núcleos y el planificador los alterna, añadiendo latencia de cambio de contexto.
- Contención de memoria (cache thrashing): demasiados hilos accediendo a la misma memoria pueden invalidar las líneas de caché mutuamente.
- Ley de Amdahl: la porción secuencial del programa (inicialización y reunión de resultados) limita la aceleración máxima teórica independientemente del número de hilos.

Ejercicio III

El programa procesa instrucciones de 8 bits (7 bits de datos + 1 bit de paridad par). El usuario ingresa la cantidad de hilos y las instrucciones se reparten en bloques. Cada hilo valida el bit de paridad de su bloque e imprime un resumen de instrucciones válidas e inválidas.

Preguntas del Ejercicio III

1. ¿Qué ventaja tiene asignar un bloque de instrucciones a cada hilo en lugar de crear un hilo por cada instrucción?

Crear un hilo tiene un costo fijo de recursos del SO (pila, PCB, registros). Si hay miles de instrucciones y se crea un hilo por cada una, se generan demasiados hilos, excediendo rápidamente los límites del sistema. Con bloques:

- Menor overhead total: se crean solo k hilos (configurable), independientemente del número de instrucciones.
- Mejor escalabilidad: el programa funciona igual con 10 instrucciones que con 10,000,000.
- Control explícito del paralelismo: se puede ajustar k al número de núcleos disponibles para lograr paralelismo óptimo.
- Reduce contención: menos hilos activos simultáneamente significa menos competencia por CPU y caché.

2. ¿Qué errores podrían ocurrir si varias instrucciones compartieran variables globales sin control?

Al acceder múltiples hilos a variables globales sin sincronización se producen condiciones de carrera (race conditions):

- Resultados incorrectos: si dos hilos leen y modifican un contador global al mismo tiempo, la operación `contador++` (read-modify-write, no atómica) puede perderse. Por ejemplo: ambos leen 5, calculan 6, escriben 6 — el contador debería ser 7.
- Comportamiento no determinista: el resultado depende del planificador del SO (timing), haciendo que el programa produzca resultados distintos en cada ejecución.
- Corrupción de datos: si la variable compartida es una estructura compleja, escrituras concurrentes sin mutex pueden dejarla en estados inválidos.
- Solución: usar `pthread_mutex_t` para proteger secciones críticas, o usar variables locales por hilo y agregar resultados solo en el main tras `pthread_join` (como se hace en este laboratorio).

Captura de funcionamiento del programa

```
CC3086 - Lab 08: Introduccion a Pthreads
Universidad del Valle de Guatemala
EJERCICIO I: Ordenamiento de matrices con hilos

[Main] Verificacion - Primeras 3 filas (primeros 10 elementos):
Fila 0: 351 974 13 856 430 878 785 300 32 829 ...
Fila 1: 457 693 5 117 333 996 540 9 986 771 ...
Fila 2: 598 693 10 721 254 997 194 978 608 538 ...

[Ejercicio I] Completado con 4 hilos.
EJERCICIO II: Calculo Paralelo (suma y pares)
Ingrese el tamaño del arreglo: 5
Ingrese la cantidad de hilos: 5

Arreglo generado: 59 82 79 61 0

[Main] Suma total del arreglo: 0
[Main] Cantidad total de numeros pares: 0

[Ejercicio II] Completado con 5 hilos.
EJERCICIO III: Decodificador de instrucciones binarias
Instrucciones cargadas: 12
Ingrese la cantidad de hilos (1-12): 5

Usando 5 hilos para 12 instrucciones.

[Main] RESUMEN GLOBAL:
Total instrucciones procesadas: 12
Total VALIDAS (paridad par OK): 0
Total INVALIDAS (paridad errada): 0

[Ejercicio III] Completado con 5 hilos.
Laboratorio 08 finalizado.

Normal program termination. Exit status: 0
```
